

Entregable Desarrollador

Dependencia (Origen → Destino)	¿Por qué existe?	Servicio / Interfaz que utiliza
Interfaz Web → Autenticación	Para proteger las rutas y asegurar que el usuario tenga el JWT (JSON Web Token) válido antes de ver datos.	<code>IAuthService.validateToken()</code>
Gestión de Expedientes → Validación Binaria	Para evitar que archivos corruptos o maliciosos lleguen al almacenamiento final. Es el "filtro" de seguridad técnica.	<code>IFileValidator.checkMagicNumbers()</code>
Gestión de Expedientes → Object Store	Para separar la persistencia de metadatos (SQL) de la persistencia binaria (PDF/IMG).	<code>IBlobStorage.uploadStream()</code>

Evaluación Administrativa → Seguimiento/Alertas	Para que el cambio de estatus de un documento dispare una notificación automática al aspirante sin bloquear el flujo principal.	INotificationService.pushAlert()
Componentes de Lógica → Base de Datos SQL	Para persistir estados, roles, logs de auditoría y relaciones relacionales.	IDataContext (Entity Framework / Hibernate)

2. Flujo de Interacción entre Componentes

A continuación, modelo los dos flujos de trabajo más importantes del sistema para visualizar la cadena de llamadas:

Flujo A: Carga de Documento por el Aspirante (RF-02)

Este flujo es de alta criticidad por el manejo de archivos binarios:

1. **Interfaz Web** (Frontend) captura el archivo.
2. **Interfaz Web** → envía a **Gestión de Expedientes** (API).
3. **Gestión de Expedientes** → invoca a **Validación Binaria** (Service).
4. **Validación Binaria** → retorna `is_valid: true`.
5. **Gestión de Expedientes** → envía flujo a **Object Store** (Repository).
6. **Gestión de Expedientes** → registra metadatos en **Base de Datos SQL**.
7. **Gestión de Expedientes** → notifica a **Seguimiento/Alertas** para actualizar el Dashboard.

Flujo B: Evaluación y Rechazo con Observación (RF-04)

1. **Interfaz Web** (Vista Admin) → **Evaluación Administrativa**.
2. **Evaluación Administrativa** → actualiza estatus en **Base de Datos SQL**.
3. **Evaluación Administrativa** → **Seguimiento/Alertas** detecta el cambio de estatus.
4. **Seguimiento/Alertas** → genera notificación para la **Interfaz Web** (Vista Aspirante).

3. Identificación de Posibles Problemas Técnicos

Como desarrollador, he identificado tres "puntos calientes" donde la arquitectura de componentes podría enfrentar desafíos:

- **Latencia en la Validación Binaria:** Si el componente de **Validación Binaria** realiza un escaneo exhaustivo de archivos muy pesados, podría bloquear el hilo de ejecución de **Gestión de Expedientes**, provocando un *Timeout* en la Interfaz Web.
 - *Solución técnica:* Implementar la validación de forma asíncrona mediante colas de mensajes (ej. RabbitMQ o procesos en segundo plano).
- **Cuellos de Botella en el Object Store:** Si el **Repositorio Object Store** (nube o servidor local) experimenta lentitud, el sistema de **Gestión de Expedientes** podría quedarse con conexiones abiertas a la **Base de Datos SQL**, agotando el pool de conexiones.
 - *Solución técnica:* Implementar el patrón *Circuit Breaker* para manejar caídas del almacenamiento sin tirar abajo la base de datos.
- **Inconsistencia de Estados (Race Conditions):** Si dos administradores intentan evaluar el mismo expediente simultáneamente, el componente de **Evaluación Administrativa** podría generar estados inconsistentes si no se maneja concurrencia optimista.
 - *Solución técnica:* Uso de versiones (*RowVersion/Timestamp*) en la capa de **Base de Datos SQL** para rechazar la segunda transacción concurrente.

Conclusión del Desarrollador: La estructura es sólida siempre que mantengamos las interfaces bien definidas. El desacoplamiento entre el almacenamiento de archivos y la base de datos relacional es nuestra mejor defensa contra problemas de escalabilidad.